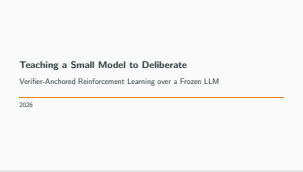
Teaching a Small Model to Deliberate

Verifier-Anchored Reinforcement Learning over a Frozen LLM

2026



This is a reinforcement-learning talk. The central object is an agent with a small action-value function over a structured state, trained against rewards a deterministic verifier supplies. Keep that frame: everything else, MCTS included, is machinery in service of learning a small, fast policy.

Motivation

Motivation

The pain: cheap models, expensive reasoning

- Small 1–7B models are fast and deployable, but lag on GSM8K and MATH
- The gap is widest exactly where answers are checkable by a non-LLM verifier
- Test-time search (best-of-N, self-consistency, MCTS) helps, but is amnesiac: every query pays full cost again, nothing is distilled back
- The question: can we close the gap *without growing the model*?

The pain: cheap models, expensive reasoning
- Small 1–7B models are fast and deployable, but lag on GSM8K and MATH
- The gap is widest exactly where answers are checkable by a non-LLM verifier
- Test-time search (best-of-N, self-consistency, MCTS) helps, but is amnesiac: every query pays full cost again, nothing is distilled back
- The question: can we close the gap *without growing the model*?

Small models are what we want to deploy, but they reason poorly across steps. Inference-time search patches the symptom and never learns from it. We want to turn that search into a teacher and learn a small policy, paying the search cost once during training rather than forever at inference.

And learned rewards get gamed

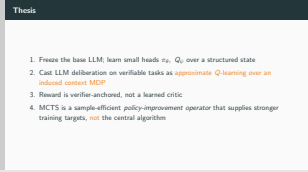
- Dense *learned* reward models are seductive but open a Goodhart attack surface
- Under enough optimization pressure, the policy converges on the proxy's defects
- Our anchor: a deterministic non-LLM verifier (SymPy equality, unit tests, Lean type-check)
- A signal enters the reward channel only if it is an objective proposition a program can check

And learned rewards get gamed
- Dense *learned* reward models are seductive but open a Goodhart attack surface
- Under enough optimization pressure, the policy converges on the proxy's defects
- Our anchor: a deterministic non-LLM verifier (SymPy equality, unit tests, Lean type-check)
- A signal enters the reward channel only if it is an objective proposition a program can check

The core reward-design commitment is an asymmetry: only verifier-checkable propositions may form rewards and Bellman backups; learned-aesthetic classifiers may enter only as search priors, never as reward. Keeping reward verifier-anchored rather than learned reduces the Goodhart attack surface by construction.

Thesis

1. Freeze the base LLM; learn small heads π_θ, Q_ψ over a structured state
2. Cast LLM deliberation on verifiable tasks as approximate Q-learning over an induced context MDP
3. Reward is verifier-anchored, not a learned critic
4. MCTS is a sample-efficient *policy-improvement operator* that supplies stronger training targets, not the central algorithm



The whole talk in one slide. The learnable object is the agent and its action-value function, trained by approximate Q-learning of the DQN lineage. Tree search is one mechanism for producing better samples and stronger targets; it is subordinate to the learning. Because the LLM is frozen, any measured improvement is attributable to the heads and the search procedure.

The setup: an induced context MDP

The setup: an induced context MDP

The induced context MDP

- State s: a structured reasoning context (not raw token soup)
- Actions A: seven coarse cognitive verbs, $|A| = 7$
- Transition P: frozen LLM + prompt template samples the next state; non-enumerable
- Reward R: ground-truth terminal $+1/-1$ from the verifier; small capped intermediate shaping
- Controller observes only *sampled* transitions: model-free RL with the LLM as the environment

The induced context MDP
- State s: a structured reasoning context (not raw token soup)
- Actions A: seven coarse cognitive verbs, $|A| = 7$
- Transition P: frozen LLM + prompt template samples the next state; non-enumerable
- Reward R: ground-truth terminal $+1/-1$ from the verifier; small capped intermediate shaping
- Controller observes only *sampled* transitions: model-free RL with the LLM as the environment

The next state is whatever the LLM continuation plus a lossy parser produce, so the transition kernel is neither enumerable nor closed-form. The controller never sums over next states; it only observes samples. That single fact later forces sampled chance nodes as the only honest estimator of the Bellman inner expectation.

Structured state: a shell the LLM fills

- Slots: problem P, Facts, open/closed Subgoals, Hypotheses, verifier log, committed answer, depth
- Load-bearing split: **Fact** ("I have proven this") vs **Hypothesis** ("I think this is true")
- Avoids two failure modes: raw tokens (no inductive bias) and one opaque embedding (no action masking)
- Serialized with structural markers: `[FACT]`, `[HYPOTHESIS]`, `[VERIFIER_CALL]`, `[STATE_END]`

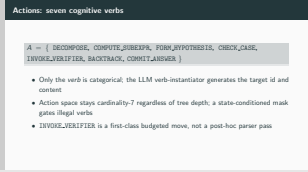
Structured state: a shell the LLM fills
- Slots: problem P, Facts, open/closed Subgoals, Hypotheses, verifier log, committed answer, depth
- Load-bearing split: **Fact** ("I have proven this") vs **Hypothesis** ("I think this is true")
- Avoids two failure modes: raw tokens (no inductive bias) and one opaque embedding (no action masking)
- Serialized with structural markers: `[FACT]`, `[HYPOTHESIS]`, `[VERIFIER_CALL]`, `[STATE_END]`

The schema makes the Markov bet explicit. Fact-versus-Hypothesis is the most important design choice: tentative claims kept separate from verified ones, which is what deliberation needs. The state is embedded as the frozen LLM's hidden state at the [STATE_END] sentinel.

Actions: seven cognitive verbs

$A = \{$ DECOMPOSE, COMPUTE_SUBEXPR, FORM_HYPOTHESIS, CHECK_CASE, INVOKE_VERIFIER, BACKTRACK, COMMIT_ANSWER $\}$

- Only the *verb* is categorical; the LLM verb-instantiator generates the target id and content
- Action space stays cardinality-7 regardless of tree depth; a state-conditioned mask gates illegal verbs
- INVOKE_VERIFIER is a first-class budgeted move, not a post-hoc parser pass

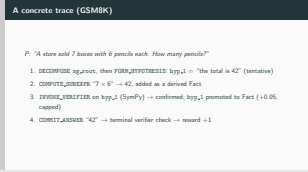


Verbs are mostly realized as prompting templates the frozen base model instantiates, but not necessarily only prompts. Keeping the verb categorical gives the Q-head a fixed output and search a stable branching factor. Making InvokeVerifier a callable verb makes the verifier first-class deliberation the agent learns to budget.

A concrete trace (GSM8K)

P: "A store sold 7 boxes with 6 pencils each. How many pencils?"

1. `DECOMPOSE sg_root`, then `FORM_HYPOTHESIS`: `hyp_1` = "the total is 42" (tentative)
2. `COMPUTE_SUBEXPR` "7×6" $\rightarrow$ 42, added as a derived Fact
3. `INVOKE_VERIFIER` on `hyp_1` (SymPy) $\rightarrow$ confirmed; `hyp_1` promoted to Fact ($+0.05$, capped)
4. `COMMIT_ANSWER` "42" $\rightarrow$ terminal verifier check $\rightarrow$ reward $+1$



This single trace is the entire RL story made concrete: state, verb-actions, sampled transitions, verifier reward. The small intermediate reward for a first-time successful verifier promotion is hard-capped so it cannot be farmed above the terminal magnitude. At inference $B{=}0$ walks this greedily; $B{>}0$ runs MCTS to refine the verb choices.

The learner

The learner

Architecture: frozen LLM, tiny trainable heads

- One frozen base LLM is *both* encoder and stochastic transition/generator (one forward, not two)
- State embedding = hidden state at `[STATE_END]`; a small adapter MLP
- Multi-head $Q_\psi(s, a)$ over the 7 verbs, plus a policy-prior head π_θ sharing the adapter
- Total trainable params under 5M: 2–3 orders of magnitude smaller than the frozen LLM
- Approximate Q-learning: replay buffer, target net synced every N_{sync}, prioritized by TD error

Architecture: frozen LLM, tiny trainable heads
- One frozen base LLM is *both* encoder and stochastic transition/generator (one forward, not two)
- State embedding = hidden state at `[STATE_END]`; a small adapter MLP
- Multi-head $Q_\psi(s, a)$ over the 7 verbs, plus a policy-prior head π_θ sharing the adapter
- Total trainable params under 5M: 2–3 orders of magnitude smaller than the frozen LLM
- Approximate Q-learning: replay buffer, target net synced every N_{sync}, prioritized by TD error

A single LLM forward yields both the state embedding and the verb-instantiator's continuation, which makes the shared-encoder design cheap. We train only the adapter and heads. Because the base never moves, representation drift is bounded and any gain is attributable to heads and search, not to fine-tuning the LLM.

Offline by design: learn from any trajectory

- Q-learning is off-policy: the max bootstrap refers to the greedy continuation, not the behavior policy
- So the learner can train from *any* trajectories covering the state-action space
- Sources: its own runs, other models' runs, logged production data, human demonstrations
- Decouples data collection from learning: a genuine strength, not a limitation
- Q_{MCTS} targets are stored *with* each replay sample, so stale entries do not break correctness

Offline by design: learn from any trajectory
- Q-learning is off-policy: the max bootstrap refers to the greedy continuation, not the behavior policy
- So the learner can train from *any* trajectories covering the state-action space
- Sources: its own runs, other models' runs, logged production data, human demonstrations
- Decouples data collection from learning: a genuine strength, not a limitation
- Q_{MCTS} targets are stored *with* each replay sample, so stale entries do not break correctness

Off-policy learning is what makes MCTS-guided collection, or any other collection, compatible with the learning rule. The quiet superpower: data source and learner are decoupled. It also sets up online continual learning, since deployment traffic is just one more source of off-policy experience.

MCTS as a policy-improvement operator (not the star)

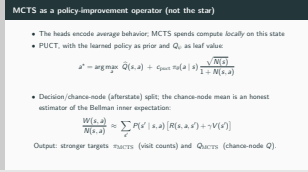
- The heads encode *average* behavior; MCTS spends compute *locally* on this state
- PUCT, with the learned policy as prior and Q_ψ as leaf value:

$$a^\star = \arg\max_a \; \widehat{Q}(s,a) \; + \; c_{\text{puct}}\, \pi_\theta(a \mid s)\, \frac{\sqrt{N(s)}}{1+N(s,a)}$$

- Decision/chance-node (afterstate) split; the chance-node mean is an honest estimator of the Bellman inner expectation:

$$\frac{W(s,a)}{N(s,a)} \;\approx\; \sum_{s'} P(s' \mid s,a)\,\big[R(s,a,s') + \gamma V(s')\big]$$

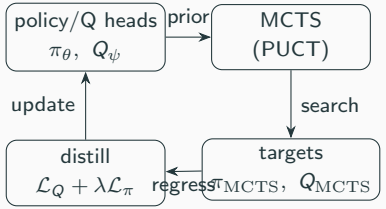
Output: stronger targets π_{MCTS} (visit counts) and Q_{MCTS} (chance-node Q).

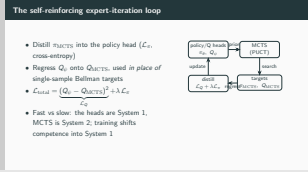


MCTS enters as a policy-improvement operator: given current estimates it produces locally improved ones at a query state, replacing the intractable Bellman sum with sampled rollouts. The sampled chance node is the only available estimator of the inner expectation. The afterstate bifurcation is borrowed (Stochastic MuZero), not claimed as novel.

The self-reinforcing expert-iteration loop

- Distill π_{MCTS} into the policy head ($\mathcal{L}_\pi$, cross-entropy)
- Regress Q_ψ onto Q_{MCTS}, used *in place of* single-sample Bellman targets
- $\mathcal{L}_{\mathrm{total}} = \underbrace{(Q_\psi - Q_{\mathrm{MCTS}})^2}_{\mathcal{L}_Q} + \lambda\,\mathcal{L}_\pi$
- Fast vs slow: the heads are System 1, MCTS is System 2; training shifts competence into System 1





The same approximate Q-learning update runs; only the targets get better, because they average over search-guided trajectories instead of single samples. This is AlphaZero's expert-iteration structure, inverted: the agent and its Q are the center, search is subordinate machinery for sample efficiency.

Reward hacking: a verifier is necessary, not sufficient

- Goodhart is the classic RL failure: optimize a mis-specified reward, get gamed behavior that scores well and reasons badly
- A non-LLM verifier removes the *learned*-reward attack surface, but a poorly specified verifier reward is still gameable
- We red-teamed our own intermediate shaping reward and found a farm: verify content-free tautologies or self-authored claims for free reward

Fix by construction, not by patch

Tie the rewarded check to the target claim; reward only the *first* success per target; hard-cap cumulative intermediate reward at $\frac{1}{5}$ of the terminal magnitude $\Rightarrow$ the ground-truth terminal verifier always dominates, so farming is *bounded*, not merely discouraged.

Reward hacking: a verifier is necessary, not sufficient
• Goodhart is the classic RL failure: optimize a mis-specified reward, get gamed behavior that scores well and reasons badly
• A non-LLM verifier removes the learned-reward attack surface, but a poorly specified verifier reward is still gameable
• We red-teamed our own intermediate shaping reward and found a farm: verify content-free tautologies or self-authored claims for free reward
Fix by construction, not by patch
Tie the rewarded check to the target claim; reward only the first success per target; hard-cap cumulative intermediate reward at 1/5 of the terminal magnitude ⇒ the ground-truth terminal verifier always dominates, so farming is bounded, not merely discouraged.

The honest slide. A verifiable reward is necessary but not sufficient: a badly specified shaping term is still hackable. We caught exactly this via adversarial review, the agent earning the small reward by verifying trivially true but irrelevant self-authored claims. The fix is structural: tie to target, reward each target once, cap the cumulative shaping well below the terminal so no farm can rival actually solving. Red-teaming the reward is part of the method.

Inference and deployment

The compute knob B: fast by default, think harder on demand

$B = 0$ **(fast)**	$B > 0$ **(search)**
masked arg max over Q_ψ; one LLM forward per decision; constant latency	run PUCT MCTS for B simulations; sample the visit-count policy

- Same trained Q_ψ and π_θ in both modes; one dial traces a compute-vs-accuracy frontier, no retraining
- Fast mode is competitive precisely because the heads were distilled from MCTS targets

The compute knob B: fast by default, think harder on demand
$B = 0$ (fast) masked arg max over Q_ψ; one LLM forward per decision; constant latency
$B > 0$ (search) run PUCT MCTS for B simulations; sample the visit-count policy
- Same trained Q_ψ and π_θ in both modes; one dial traces a compute-vs-accuracy frontier, no retraining
- Fast mode is competitive precisely because the heads were distilled from MCTS targets

Two modes, not two algorithms: the same learned actor with different amounts of search on top. The search budget becomes a continuous dial the user controls at inference. Because $Q_\psi \approx Q_{\mathrm{MCTS}}$ and $\pi_\theta \approx \pi_{\mathrm{MCTS}}$ at the training budget, B=0 already carries much of the benefit.

Online continuous learning in deployment

- Because it is off-policy and feedback-anchored, learning need not stop at deploy
- Any verifier-style signal keeps it improving, including user thumbs up/down
- The signal must stay a *structural check*, never a learned classifier in the Bellman target
- Adapter-tier separation: per-user adapters absorb idiosyncratic feedback; the global adapter accepts only changes that survive aggregation

A model that gets better the longer it is used.

Online continuous learning in deployment
- Because it is off-policy and feedback-anchored, learning need not stop at deploy
- Any verifier-style signal keeps it improving, including user thumbs up/down
- The signal must stay a *structural check*, never a learned classifier in the Bellman target
- Adapter-tier separation: per-user adapters absorb idiosyncratic feedback; the global adapter accepts only changes that survive aggregation

A model that gets better the longer it is used.

The small adapter MLP is the plug-in point for deployment learning. The tiering keeps any one user's signals from dominating global updates. The verifier stays the trusted anchor throughout, which bounds Goodhart drift even as feedback arrives from the wild.

Positioning and evaluation

Positioning and defensible novelty

- **Search-and-learning in games** (AlphaZero, expert iteration): we invert it, agent and Q_ψ central, search optional
- **Verifier-anchored reasoning** (PRMs, AlphaCode, AlphaProof, DeepSeek-R1): the verifier-vs-aesthetic asymmetry is load-bearing; `INVOKE_VERIFIER` is a callable verb
- **VerifierQ**: verifier-as-*action* not verifier-as-Q; online distilled targets not fixed-dataset fitting; categorical multi-head Q
- **rStar-Math**: same family, but ours is a structured-state seven-verb MDP
- **vs STaR / ReST-EM / Math-Shepherd**: we distill per-step Q_{MCTS} as a genuine Bellman target, not trajectory-level or a process-reward model

Positioning and defensible novelty
- **Search-and-learning in games** (AlphaZero, expert iteration): we invert it, agent and Q_ψ central, search optional
- **Verifier-anchored reasoning** (PRMs, AlphaCode, AlphaProof, DeepSeek-R1): the verifier-vs-aesthetic asymmetry is load-bearing; INVOKE_VERIFIER is a callable verb
- **VerifierQ**: verifier-as-action not verifier-as-Q; online distilled targets not fixed-dataset fitting; categorical multi-head Q
- **rStar-Math**: same family, but ours is a structured-state seven-verb MDP
- **vs STaR / ReST-EM / Math-Shepherd**: we distill per-step Q_{MCTS} as a genuine Bellman target, not trajectory-level or a process-reward model

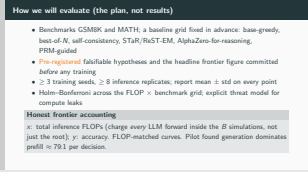
We inherit AlphaZero's joint search-and-learning structure but invert its center, making non-search baselines first-class. Against the closest Q-learning core, VerifierQ, we differ on three concrete axes. We do not claim the afterstate bifurcation as novel; novelty is its application to the non-enumerable LLM-induced transition.

How we will evaluate (the plan, not results)

- Benchmarks GSM8K and MATH; a baseline grid fixed in advance: base-greedy, best-of-N, self-consistency, STaR/ReST-EM, AlphaZero-for-reasoning, PRM-guided
- Pre-registered falsifiable hypotheses and the headline frontier figure committed *before* any training
- ≥ 3 training seeds, ≥ 8 inference replicates; report mean $\pm$ std on every point
- Holm–Bonferroni across the FLOP $\times$ benchmark grid; explicit threat model for compute leaks

Honest frontier accounting

x: total inference FLOPs (charge *every* LLM forward inside the B simulations, not just the root); y: accuracy. FLOP-matched curves. Pilot found generation dominates prefill $\approx$ 79:1 per decision.



Methodology, not a victory lap. We deliberately do not lean on current pilot numbers; strong empirical results are in progress. Pre-registration, seeds, and the correction criterion are committed up front so the eventual claim is falsifiable rather than fitted after the fact. Honest accounting is what makes "we beat best-of-N" mean "we learned something," not "we spent more compute."

Vision

A small, fast model that learns to deliberate over structured states,
anchored by verifiable (non-learned) rewards.

Offline today, online-continual in deployment,
with a single test-time compute knob.

MCTS earns its keep only as the operator that manufactures stronger targets.
Next: run the pre-registered frontier at scale and report it honestly.

Vision
A small, fast model that learns to deliberate over structured states,
anchored by verifiable (non-learned) rewards.
Offline today, online-continual in deployment,
with a single test-time compute knob.
MCTS earns its keep only as the operator that manufactures stronger targets.
Next: run the pre-registered frontier at scale and report it honestly.

The takeaway is the RL-first frame: we learn a small policy and Q over a structured state with verifiable rewards; search is the sample-efficiency machinery that teaches the heads. The same artifact serves fast by default and thinks harder on demand, and keeps learning after deployment. Next step: the rigorous pre-registered study.